

Mini Project 4. Simulasi dan Karakterisasi Channel Akustik

Mini Project 4 melanjutkan langsung Mini Project 3. Mahasiswa tidak melakukan pengukuran fisik ruangan. Sebagai gantinya, mahasiswa membuat satu respons impuls sintetis yang reproducible, mengonvolusikannya dengan audio referensi, menghitung respons frekuensi dan energy decay curve, lalu menyimpan artefak channel untuk Mini Project 5. Pengukuran MLS dan swept sine tetap menjadi materi inti bab dan dapat dilakukan sebagai praktikum lanjutan di luar rangkaian mini project 15–30 menit.

Tujuan dan Handoff dari Mini Project 3

Sebelum mulai, `project_status.json` harus berisi status `ready_for_mp04`. Mini Project 4 memakai `data/raw/nyala_001.wav` dan `config/spectral_config.json`. File `features/intermediate/mp03_reference_stft.npz` digunakan sebagai bukti bahwa baseline spektral telah selesai dan tidak perlu dihitung ulang dari nol sebagai konfigurasi baru.

Hasil Wajib Mini Project 4

- `config/channel_config.json` berisi parameter respons impuls sintetis dan faktor normalisasi output.
- `features/intermediate/mp04_synthetic_rir.npz` berisi respons impuls, sumbu waktu, respons frekuensi, dan energy decay curve.
- `data/processed/mp04_channel/nyala_001_channel.wav` berisi audio referensi setelah channel.
- `results/figures/mp04_channel_characterization.png` berisi respons impuls, respons magnitudo, dan decay curve.
- `results/figures/mp04_clean_vs_channel.png` membandingkan waveform serta spektrum audio bersih dan channel-affected.
- `results/metrics/mp04_channel_metrics.json` berisi estimasi RT60, peak output, durasi IR, dan perubahan spectral distance sederhana.
- `project_status.json` diperbarui menjadi `ready_for_mp05`.

Perangkat dan Perangkat Lunak

Gunakan environment Python yang sama seperti Mini Project 1–3. Paket yang digunakan hanya NumPy, SciPy, Matplotlib, dan modul standar Python. Tidak diperlukan speaker, mikrofon tambahan, atau koneksi internet.

Prosedur Langkah demi Langkah

Langkah 1. Periksa Handoff Mini Project 3

Buka terminal pada folder `audio_ai_embedded_project` dan pastikan file berikut ada:

```
project_status.json
config/spectral_config.json
data/raw/nyala_001.wav
features/intermediate/mp03_reference_stft.npz
```

Buka `project_status.json`. Nilai status wajib `ready_for_mp04`. Jika tidak, jangan membuat file baru; selesaikan Mini Project 3 terlebih dahulu.

Langkah 2. Buat Skrip Mini Project 4

Buat file `mini_project_04.py` di folder utama proyek. Kode berikut membangun respons impuls sintetis dengan direct path, tiga early reflections, dan reverberation tail yang dapat direproduksi melalui random seed tetap.

```
from pathlib import Path
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import fftconvolve

ROOT = Path(__file__).resolve().parent
STATUS_PATH = ROOT / "project_status.json"
SPECTRAL_PATH = ROOT / "config/spectral_config.json"
INPUT_WAV = ROOT / "data/raw/nyala_001.wav"
MP03_STFT = ROOT / "features/intermediate/mp03_reference_stft.npz"

# 1. Validasi handoff Mini Project 3
with open(STATUS_PATH, "r", encoding="utf-8") as f:
    status = json.load(f)
if status.get("status") != "ready_for_mp04":
    raise RuntimeError("Mini Project 3 belum berstatus ready_for_mp04")

with open(SPECTRAL_PATH, "r", encoding="utf-8") as f:
    spectral = json.load(f)
if not MP03_STFT.exists():
    raise FileNotFoundError("Artefak STFT Mini Project 3 tidak ditemukan")

fs, raw = wavfile.read(INPUT_WAV)
if raw.ndim > 1:
    raw = raw.mean(axis=1)
if np.issubdtype(raw.dtype, np.integer):
    scale = float(max(abs(np.iinfo(raw.dtype).min), np.iinfo(raw.dtype).max))
    x = raw.astype(np.float32) / scale
else:
    x = raw.astype(np.float32)

if int(fs) != int(spectral["sample_rate_hz"]):
    raise ValueError("Sampling rate tidak konsisten dengan spectral_config.json")

# 2. Buat respons impuls sintetis yang reproducible
rng = np.random.default_rng(2026)
ir_duration_s = 0.35
rir = np.zeros(int(round(ir_duration_s * fs)), dtype=np.float32)
rir[0] = 1.0

reflection_delays_ms = [16, 34, 58]
reflection_gains = [0.42, -0.27, 0.18]
for delay_ms, gain in zip(reflection_delays_ms, reflection_gains):
    idx = int(round(delay_ms * fs / 1000.0))
    rir[idx] += gain

# Reverberation tail: random noise dengan envelope eksponensial
rt60_target_s = 0.35
tail_start_s = 0.075
tail_start = int(round(tail_start_s * fs))
t_tail = np.arange(len(rir) - tail_start) / fs
# Amplitudo turun 60 dB = faktor 0.001 pada rt60_target_s
amp_env = np.exp(np.log(0.001) * t_tail / rt60_target_s)
rir[tail_start:] += 0.055 * amp_env * rng.normal(size=len(t_tail))

# 3. Konvolusi dan normalisasi agar tidak clipping
y = fftconvolve(x, rir, mode="full").astype(np.float32)
peak_before = float(np.max(np.abs(y)) + 1e-12)
target_peak = 0.95
normalization_gain = target_peak / peak_before
y *= normalization_gain

# 4. Karakterisasi respons frekuensi dan Energy Decay Curve
nfft = int(spectral.get("nfft", 512))
nfft_ir = max(8192, 2 ** int(np.ceil(np.log2(len(rir)))))
freqs = np.fft.rfftfreq(nfft_ir, d=1.0/fs)
H = np.fft.rfft(rir, n=nfft_ir)
mag_db = 20 * np.log10(np.abs(H) / (np.max(np.abs(H)) + 1e-12) + 1e-10)

energy = np.cumsum((rir[::-1] ** 2))[:-1]
```

```

edc_db = 10 * np.log10(energy / (energy[0] + 1e-15) + 1e-15)
time_ir = np.arange(len(rir)) / fs

# Estimasi RT60 dari slope -5 sampai -35 dB bila titik cukup
mask = (edc_db <= -5.0) & (edc_db >= -35.0)
if np.count_nonzero(mask) >= 10:
    slope, intercept = np.polyfit(time_ir[mask], edc_db[mask], 1)
    rt60_est = float(-60.0 / slope) if slope < 0 else float("nan")
else:
    slope = float("nan")
    rt60_est = float("nan")

# 5. Bandingkan spektrum clean dan channel pada panjang FFT yang sama
N = min(len(x), len(y))
Nspec = 4096
X = np.abs(np.fft.rfft(x[:N], n=Nspec))
Y = np.abs(np.fft.rfft(y[:N], n=Nspec))
Xdb = 20*np.log10(X/(X.max()+1e-12)+1e-9)
Ydb = 20*np.log10(Y/(Y.max()+1e-12)+1e-9)
f_audio = np.fft.rfftfreq(Nspec, 1/fs)
spectral_distance_db = float(np.sqrt(np.mean((Xdb - Ydb)**2)))

# 6. Siapkan folder output
out_audio = ROOT / "data/processed/mp04_channel"
out_feat = ROOT / "features/intermediate"
out_fig = ROOT / "results/figures"
out_met = ROOT / "results/metrics"
for p in [out_audio, out_feat, out_fig, out_met]:
    p.mkdir(parents=True, exist_ok=True)

# Simpan WAV PCM16
wavfile.write(out_audio / "nyala_001_channel.wav", fs,
              np.int16(np.clip(y, -1, 1) * 32767))

# Simpan artefak numerik
np.savez_compressed(
    out_feat / "mp04_synthetic_rir.npz",
    rir=rir,
    time_s=time_ir,
    freqs_hz=freqs,
    magnitude_db=mag_db,
    edc_db=edc_db,
)

channel_config = {
    "sample_rate_hz": int(fs),
    "ir_duration_s": ir_duration_s,
    "reflection_delays_ms": reflection_delays_ms,
    "reflection_gains": reflection_gains,
    "tail_start_s": tail_start_s,
    "rt60_target_s": rt60_target_s,
    "random_seed": 2026,
    "output_target_peak": target_peak,
    "normalization_gain": float(normalization_gain),
}
with open(ROOT / "config/channel_config.json", "w", encoding="utf-8") as f:
    json.dump(channel_config, f, indent=2)

metrics = {
    "rt60_estimated_s": rt60_est,
    "decay_slope_db_per_s": float(slope),
    "rir_duration_s": ir_duration_s,
    "peak_before_normalization": peak_before,
    "normalization_gain": float(normalization_gain),
    "spectral_distance_db_rms": spectral_distance_db,
}
with open(out_met / "mp04_channel_metrics.json", "w", encoding="utf-8") as f:
    json.dump(metrics, f, indent=2)

# 7. Figure karakterisasi channel
fig, ax = plt.subplots(3, 1, figsize=(9, 9))
ax[0].plot(time_ir, rir, linewidth=0.8)
ax[0].set_title("Synthetic room impulse response")
ax[0].set_xlabel("Time (s)")
ax[0].set_ylabel("Amplitude")
ax[0].grid(alpha=0.2)

ax[1].plot(freqs, mag_db, linewidth=0.9)
ax[1].set_xlim(0, min(6000, fs/2))

```

```

ax[1].set_ylim(-45, 3)
ax[1].set_title("Channel magnitude response")
ax[1].set_xlabel("Frequency (Hz)")
ax[1].set_ylabel("Magnitude (dB)")
ax[1].grid(alpha=0.2)

ax[2].plot(time_ir, edc_db, linewidth=0.9)
ax[2].axhline(-5, linestyle="--", linewidth=0.8)
ax[2].axhline(-35, linestyle="--", linewidth=0.8)
ax[2].set_ylim(-65, 2)
ax[2].set_title(f"Energy decay curve | RT60 est. = {rt60_est:.3f} s")
ax[2].set_xlabel("Time (s)")
ax[2].set_ylabel("Energy (dB)")
ax[2].grid(alpha=0.2)
plt.tight_layout()
plt.savefig(out_fig / "mp04_channel_characterization.png", dpi=180)
plt.close()

# 8. Figure clean vs channel
fig, ax = plt.subplots(2, 1, figsize=(9, 6))
t = np.arange(N) / fs
show = min(N, int(0.25*fs))
ax[0].plot(t[:show], x[:show], label="clean", linewidth=0.8)
ax[0].plot(t[:show], y[:show], label="channel", linewidth=0.8, alpha=0.8)
ax[0].set_title("Waveform comparison")
ax[0].set_xlabel("Time (s)")
ax[0].legend()
ax[0].grid(alpha=0.2)
ax[1].plot(f_audio, Xdb, label="clean", linewidth=0.9)
ax[1].plot(f_audio, Ydb, label="channel", linewidth=0.9)
ax[1].set_xlim(0, min(6000, fs/2))
ax[1].set_ylim(-80, 3)
ax[1].set_title("Normalized spectrum comparison")
ax[1].set_xlabel("Frequency (Hz)")
ax[1].set_ylabel("Magnitude (dB)")
ax[1].legend()
ax[1].grid(alpha=0.2)
plt.tight_layout()
plt.savefig(out_fig / "mp04_clean_vs_channel.png", dpi=180)
plt.close()

# 9. Handoff ke Mini Project 5
status.update({
    "status": "ready_for_mp05",
    "last_completed_mini_project": 4,
    "channel_audio": "data/processed/mp04_channel/nyala_001_channel.wav",
    "channel_config": "config/channel_config.json",
    "channel_rir": "features/intermediate/mp04_synthetic_rir.npz",
    "channel_metrics": "results/metrics/mp04_channel_metrics.json",
})
with open(STATUS_PATH, "w", encoding="utf-8") as f:
    json.dump(status, f, indent=2)

print("Mini Project 4 selesai")
print("RT60 estimated (s):", round(rt60_est, 3))
print("Spectral distance RMS (dB):", round(spectral_distance_db, 3))
print("Status: ready_for_mp05")

```

Langkah 3. Jalankan Skrip

```
python mini_project_04.py
```

Skrip harus berakhir dengan pesan Mini Project 4 selesai dan status ready_for_mp05. Nilai RT60 yang diperoleh tidak harus tepat 0,35 s karena tail sintesis juga mengandung early reflections, random component, durasi IR terbatas, dan prosedur estimasi berbasis regresi.

Langkah 4. Periksa Dua Gambar Hasil

Buka results/figures/mp04_channel_characterization.png. Identifikasi direct path pada awal IR, puncak early reflections, pola respons frekuensi yang tidak rata, dan garis decay yang menurun. Kemudian buka mp04_clean_vs_channel.png dan perhatikan bahwa isi dasar sinyal tetap sama tetapi waveform serta spektrumnya berubah setelah channel.

Langkah 5. Periksa Artefak dan Handoff ke Mini Project 5

Pastikan file berikut ada:

```
config/channel_config.json
features/intermediate/mp04_synthetic_rir.npz
data/processed/mp04_channel/nyala_001_channel.wav
results/metrics/mp04_channel_metrics.json
results/figures/mp04_channel_characterization.png
results/figures/mp04_clean_vs_channel.png
project_status.json
```

Buka `project_status.json`. Status akhir harus `ready_for_mp05`. Jangan menghapus audio bersih; Mini Project 5 akan membandingkan sinyal bersih, sinyal setelah channel, dan keluaran filter.

Tabel 4.8. Kriteria lulus Mini Project 4 dan handoff ke Mini Project 5.

Artefak / pemeriksaan	Kriteria lulus	Digunakan pada MP5
<code>project_status.json</code>	<code>status = ready_for_mp05</code>	Mengaktifkan skrip Mini Project 5
<code>channel_config.json</code>	Parameter channel lengkap dan seed tersimpan	Menjaga eksperimen dapat diulang
<code>nyala_001_channel.wav</code>	Dapat diputar dan tidak clipping	Sinyal uji filter
<code>mp04_synthetic_rir.npz</code>	rir dan <code>magnitude_db</code> tersedia	Referensi karakteristik channel
<code>mp04_channel_metrics.json</code>	RT60 dan spectral distance tercatat	Baseline sebelum filtering
Gambar karakterisasi	IR, $H(f)$, EDC terbaca	Interpretasi sebelum desain filter

Batas waktu dan aturan serial

Jangan menambah eksperimen baru. Pastikan enam artefak inti berhasil dibuat dan status `ready_for_mp05` tercapai. Pengukuran ruangan fisik menggunakan MLS atau swept sine adalah praktikum lanjutan, bukan syarat untuk melanjutkan rangkaian Mini Project 1–16.

Referensi Bab 4

1. Kuttruff, H. Room Acoustics. CRC Press.
2. Müller, S., & Massarani, P. Transfer-function measurement with sweeps. Journal of the Audio Engineering Society.
3. Farina, A. Simultaneous measurement of impulse response and distortion with a swept-sine technique. AES Convention.
4. Schroeder, M. R. New method of measuring reverberation time. Journal of the Acoustical Society of America.
5. ISO 3382-1. Acoustics — Measurement of room acoustic parameters — Part 1: Performance spaces.
6. ISO 3382-2. Acoustics — Measurement of room acoustic parameters — Part 2: Reverberation time in ordinary rooms.
7. Oppenheim, A. V., & Schaffer, R. W. Discrete-Time Signal Processing. Pearson.
8. Lyons, R. G. Understanding Digital Signal Processing. Pearson.